

Reading and converting a simple configuration file with Sections and Key-Value Pairs into a structured dictionary in Python

LEX Section Commands

```
import ply.lex as lex
```

First, we import the `ply.lex` library under the name `lex`.

List of Tokens

```
tokens = (  
    'LBRACKET', # [  
    'RBRACKET', # ]  
    'EQUAL',    # =  
    'ID',       # section name or key name  
    'VALUE',    # value  
    'NEWLINE'   # to detect end of line  
)
```

Definition of Simple Tokens

```
t_LBRACKET = r'\['  
t_RBRACKET = r'\]'  
t_EQUAL    = r'='
```

Ignoring Comments (#)

```
t_ignore_COMMENT = r'\#.*'
```

Rule for ID

```
def t_ID(t):  
    r'[a-zA-Z_][a-zA-Z0-9_\-]*'  
    return t
```

Rule for VALUE

```
def t_VALUE(t):  
    r'^=\\[\]\n\r\#]+'  
    t.value = t.value.strip()  
    return t
```

Rule for NEWLINE

```
def t_NEWLINE(t):  
    r'\n+'  
    t.lexer.lineno += len(t.value)  
    return t
```

Ignoring spaces and tabs

```
t_ignore = ' \t'
```

Error handling

```
def t_error(t):  
    print(f"Lexer Error: Illegal character '{t.value[0]}' at line  
{t.lexer.lineno}")  
    t.lexer.skip(1)
```

Creating the lexer

```
lexer = lex.lex()
```

YACC Section Commands

```
import ply.yacc as yacc  
import lex
```

import the ply.yacc library under the name yacc and also
import lex as the file that we built in **LEX Section
Commands**.

Program Rule

```
def p_program(p):  
    """  
    program : statements  
    """  
    p[0] = config
```

Statements list

```
def p_statements_list(p):  
    """  
    statements : statement  
               | statements statement  
    """  
    pass
```

Section Rule

```
def p_statement_section(p):  
    '''  
    statement : LBRACKET ID RBRACKET NEWLINE  
    '''  
    current_section = p[2]  
    config[current_section] = {}
```

Key-Value Rule

```
def p_statement_key_value(p):  
    '''  
    statement : ID EQUAL VALUE NEWLINE  
    '''  
    key = p[1]  
    value = p[3]  
    if current_section not in config:  
        config[current_section] = {}  
    config[current_section][key] = value
```

Blank Newline Rule

```
def p_statement_newline_only(p):  
    '''  
    statement : NEWLINE  
    '''  
    pass
```

Syntax Error Handler

```
def p_error(p):  
    if p:  
        print(f'Syntax Error: Unexpected token '{p.value}' at line  
{p.lineno}')  
    else:  
        print("Syntax Error at end of input")
```

Building parser

```
parser = yacc.yacc()
```

Execution Method

Run the lex file by entering : `flex (lex file name or location path)`

Run the yacc file by entering : `bison -d (yacc file name or location path)`